

1 Introduction

Two galaxies passing by each other is an incredibly complex event. The gravitational pull of one galaxy on the other, and the pull of the components of the galaxies between themselves create complex structures such as tidal streams and send both galaxies into new directions. Due to the length of the event, they are also impossible to observe in their entirety. Modelling them is thus the only way to understand how they evolve before, during and after the flyby. Due to the size of the event and the space involved, it can be modelled with a simple algorithm involving only gravitational pull from particle interactions.

On the other hand, stellar collapse necessitate a more complex modellisation. Due to the speed at which the particles collapse inward, they rebound, and the star contracts and decontract. This continues indefinitely if the gravitational pull is the only effect taken into account. Thus, to accurately model a star forming, the calculation of the acceleration of the particles also need to involve the pressure and the viscosity due to neighbouring particles.

This project is composed of two parts. In the first part, we will implement a numerical model of a moving lower mass galaxy flying by a stationary higher mass galaxy. In the second part, we will model the collapse of a star. Both models use N-body simulations, with leap-frogs algorithms to simulate the evolution of both processes as multiple particles moving relatively to each others across time.

2 Implementation

Both models were based on the same structure. A main file called forward multiple functions to initiate, evolve and plot the galaxies and stars. The first set of functions placed N particles in a circle around a center of mass (x, y) with a normal distribution for the radius, and a uniform distribution for the angle. It calculated an initial velocity in the x and y direction using 0.4 of a Keplerian velocity, which is dependent on the radius from the center of mass. Then the acceleration of each particle was calculated from this initial velocity and the position of all the other particles in the simulation. A second set of functions updated the positions, velocity and acceleration of the particles after a time step dt using a leapfrog algorithm. It first calculated an updated velocity after half a step using the previous acceleration, then calculated an updated position using this new velocity, before getting the updated acceleration and taking the second half-step in velocity using this new acceleration. A last set of function created an updated the plots, making a series of image every few time steps.

2.1 Equation of motion

The equation of motion was the most complex part of this model, as it had to be calculated individually for each particle. It was present in two different ways between the two models, and this was the largest difference between them. One of the model only included gravitational effects on the motion of the particles, while the other one included both the gravitational effects, the pressure gradient, and the effects of viscosity. The gravitational effects were a weighted sum of the pull of the other particles as a function of their mass and distance to the particle of interest:

$$\vec{F}_j = m_j G \sum_{i \neq j} \frac{m_i}{r_{ji}^3} \vec{r}_{ij} = m_j \vec{a}_j \Leftrightarrow \vec{a}_j = G \sum_{i \neq j} \frac{m_i}{r_{ji}^3} \vec{r}_{ij}.$$

The pressure gradient was slightly more complex, as it needs to be calculated as a function of the average density around the particle of interest instead of its interaction with the other individual particles. For this, we define a density kernel at each point in space which depends on the presence of other particles within the vicinity. The density at a location $\vec{r}$ is:

$$\rho(\vec{r}) = \sum_i m_i W_i(|\vec{r} - \vec{r}_i|, h).$$

Where:

$$W_i(q, h) = C \begin{cases} (2 - q)^3 - 4(1 - q^3) & \text{if } 1 > q \geq 0, \\ (2 - q)^3 & \text{if } 2 > q \geq 1, \\ 0 & \text{if } q \geq 2. \end{cases} \quad (1)$$

And $C = \frac{5}{14\pi h^2}$, $q_i = \frac{|\vec{r} - \vec{r}_i|}{h}$ and h is a constant which we chose to be $0.02 \cdot R_\odot$. From this, assuming an isothermal equation of state, it is possible to obtain the pressure : $P = \kappa \rho^2$, and from it, the pressure gradient:

$$\vec{\nabla} P(\vec{r}_j) = 2\kappa \sum_i m_i \vec{\nabla}_j W(|\vec{r}_j - \vec{r}_i|, h)$$

Finally, the impact of viscosity is simply a constant ν multiplied by the speed of the particle j at time t .

2.2 Acceleration of the code

As an N-body simulation is extremely taxing for the machine running it, multiple methods were used to accelerate the processus, and make it less taxing. There were two main possible path. The first one was to use the Numba package and compile the Python code into a language which the machine has less difficulty running. It also included the parallelisation of some parts of the code. In particular, for loops are extremely slow in Python. By running multiple of the item parallelly, it accelerated the code significantly. The second method was to entirely erase the need for for loops by using NumPy arrays.

The code the need for loops is reproduced here:

```

p_sub_0 = np.repeat(p[0,:], len(p[0,:]), axis=0).reshape(len(p[0,:]),
len(p[0,:]))
p_sub_1 = np.repeat(p[1,:], len(p[1,:]), axis=0).reshape(len(p[1,:]),
len(p[1,:]))
r_ji_x = p[0,:].transpose()-p_sub_0
r_ji_y = p[1,:].transpose()-p_sub_1

#Clip anything too close to 0
r_ji = np.clip(np.sqrt((r_ji_x)**2 + (r_ji_y)**2), 0.1*Rg, None)

#Get the acceleration
acc[0,:] = - G * sum((mp[:] / (r_ji)**3)*r_ji_x)
acc[1,:] = - G * sum((mp[:] / (r_ji)**3)*r_ji_y)

```

It creates a larger matrix of size *number of particle* $\times$ *number of particles*, such that when calculating the resulting radius, it should produce this sequence of arrays:

$$r_{ji,x} = \begin{bmatrix} p_{x1} \\ p_{x2} \\ p_{x3} \end{bmatrix} - \begin{bmatrix} p_{x1} & p_{x2} & p_{x3} \\ p_{x1} & p_{x2} & p_{x3} \\ p_{x1} & p_{x2} & p_{x3} \end{bmatrix} = \begin{bmatrix} p_{x1} - p_{x1} & p_{x1} - p_{x2} & p_{x1} - p_{x3} \\ p_{x2} - p_{x1} & p_{x2} - p_{x2} & p_{x2} - p_{x3} \\ p_{x3} - p_{x1} & p_{x3} - p_{x2} & p_{x3} - p_{x3} \end{bmatrix}$$

Then:

$$r_{ji} = \sqrt{\begin{bmatrix} p_{x1} - p_{x1} & p_{x1} - p_{x2} & p_{x1} - p_{x3} \\ p_{x2} - p_{x1} & p_{x2} - p_{x2} & p_{x2} - p_{x3} \\ p_{x3} - p_{x1} & p_{x3} - p_{x2} & p_{x3} - p_{x3} \end{bmatrix}^2 + \begin{bmatrix} p_{y1} - p_{y1} & p_{y1} - p_{y2} & p_{y1} - p_{y3} \\ p_{y2} - p_{y1} & p_{y2} - p_{y2} & p_{y2} - p_{y3} \\ p_{y3} - p_{y1} & p_{y3} - p_{y2} & p_{y3} - p_{y3} \end{bmatrix}^2}$$

Which produces an n by n matrix where each row is the radius between a particle and all the others. And finally, the calculation of the acceleration sums over the entire row, calculating the sum of the contribution of each particle i on the acceleration of particle j.

To test the comparative speed of the three methods, I used a smaller simulation, with only 500 particles in one galaxy, and 250 in the other. The three methods being: with no acceleration, using Numba, and using array maths. The fastest was the Numba method, at 529.2 seconds, followed by the array method at 1309 seconds, more than double the time. As expected, the method with no acceleration was last, taking nearly 1500 seconds. So, while compiling the code in Numba and running multiple loops in parallel is the best option, reducing the number of for loop did significantly improve the running of the simulation. The large matrix it has to build and the plotting end up being the longer operation instead of the for loop, and they are shorter.

3 Results

3.1 Galaxy Merger

3.2 Stellar Collapse

4 Discussion

- Why did you *need* to perform a numerical simulation to solve this astrophysical problem?
- Motivate the validity of the models used in this project. What physical aspects are well captured by them? What are their limitations?
- What physical ingredients could be improved / added in order to make the models more *realistic*?